



Introduction to ML Frameworks: TensorFlow, PyTorch & Scikit-learn

Presented by

Dr. Anilkumar V. Brahmane

Computer Engineering Department
Sanjivani College of Engineering

Course Details

B.Tech Computer Engineering

Progression: Basics →

Moderate → Advanced

4 Parts | 20 Content Slides |

Python-Focused



Dr. Anilkumar V. Brahmane | Computer Engineering Dept. | Sanjivani College of Engineering

What is Machine Learning? — The Big Picture

Two Paradigms: Programming vs. Learning

Traditional Programming: The developer explicitly encodes rules. You write logic → computer follows → output is produced. Every edge case must be anticipated and coded manually.

Machine Learning: You supply data and desired outputs → the computer discovers rules automatically. The model generalises to unseen data without being explicitly programmed for each case.

Why Frameworks? Writing ML from scratch requires implementing matrix calculus, gradient descent, GPU kernel code, and memory management — a months-long effort. Frameworks give you ready-made, optimised building blocks so you focus on the problem, not the plumbing.

Analogy: Just as you write Python instead of binary machine code, you use TensorFlow/PyTorch instead of raw CUDA GPU code. Abstraction accelerates progress.

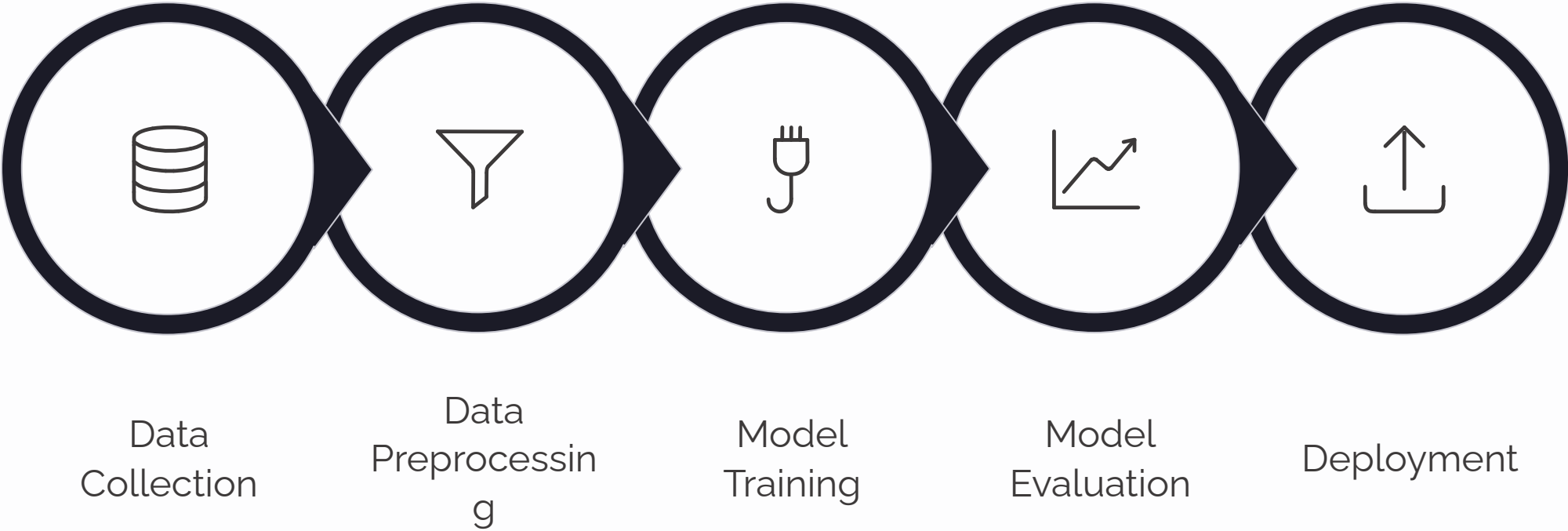
Three Pillars of Modern ML

 **Data**
Collected, cleaned, and labelled datasets

 **Model**
Framework-powered algorithm that learns patterns

 **Deployment**
Serving predictions in real-world applications

Approach	Input	Process	Output
Traditional	Data + Rules	Computer executes	Answers
Machine Learning	Data + Answers	Computer learns	Rules / Model



The Three Frameworks at a Glance — Who Made What & Why



Scikit-learn (2007)

Built by David Cournapeau; later supported by INRIA, France. Designed exclusively for **classical ML algorithms** — no deep learning. Built on NumPy and SciPy. Best for tabular/structured data, quick prototyping, and interpretable production models. The go-to default: *"If Scikit-learn solves it, stop there."*



TensorFlow (2015)

Built by **Google Brain Team**. Designed for large-scale deep learning and production-ready AI. Powers Google Search, Gmail spam filters, YouTube recommendations, and Google Translate. Offers the most comprehensive deployment ecosystem: TF Lite (mobile/edge), TF Serving (REST API), TF.js (browser).



PyTorch (2016)

Built by **Facebook's AI Research Lab (FAIR)**. Known for its dynamic computation graph ("Define-by-Run") and Pythonic design. Dominates AI research and academia — over 75% of research papers use PyTorch. Powers GPT, LLaMA, BERT, and most modern Large Language Models.

Framework	Creator	Year	Primary Use	Difficulty
Scikit-learn	Community / INRIA	2007	Classical ML	★ Beginner
TensorFlow	Google Brain	2015	Deep Learning + Production	★★★ Intermediate
PyTorch	Facebook FAIR	2016	Research + Deep Learning	★★ Beginner-Friendly



PART 1 OF 4

Scikit-learn

Start Here First — Classical ML for Structured Data

Master the fundamentals: **fit()** → **predict()** → **score()** — the universal pattern behind every ML algorithm.

□ Dr. Anilkumar V. Brahmane | Computer Engineering Dept. | Sanjivani College of Engineering

Scikit-learn Architecture — How It Is Built

Core Design Philosophy

Consistent, Simple API: Every algorithm — whether Logistic Regression or Random Forest — follows the exact same three-step pattern:

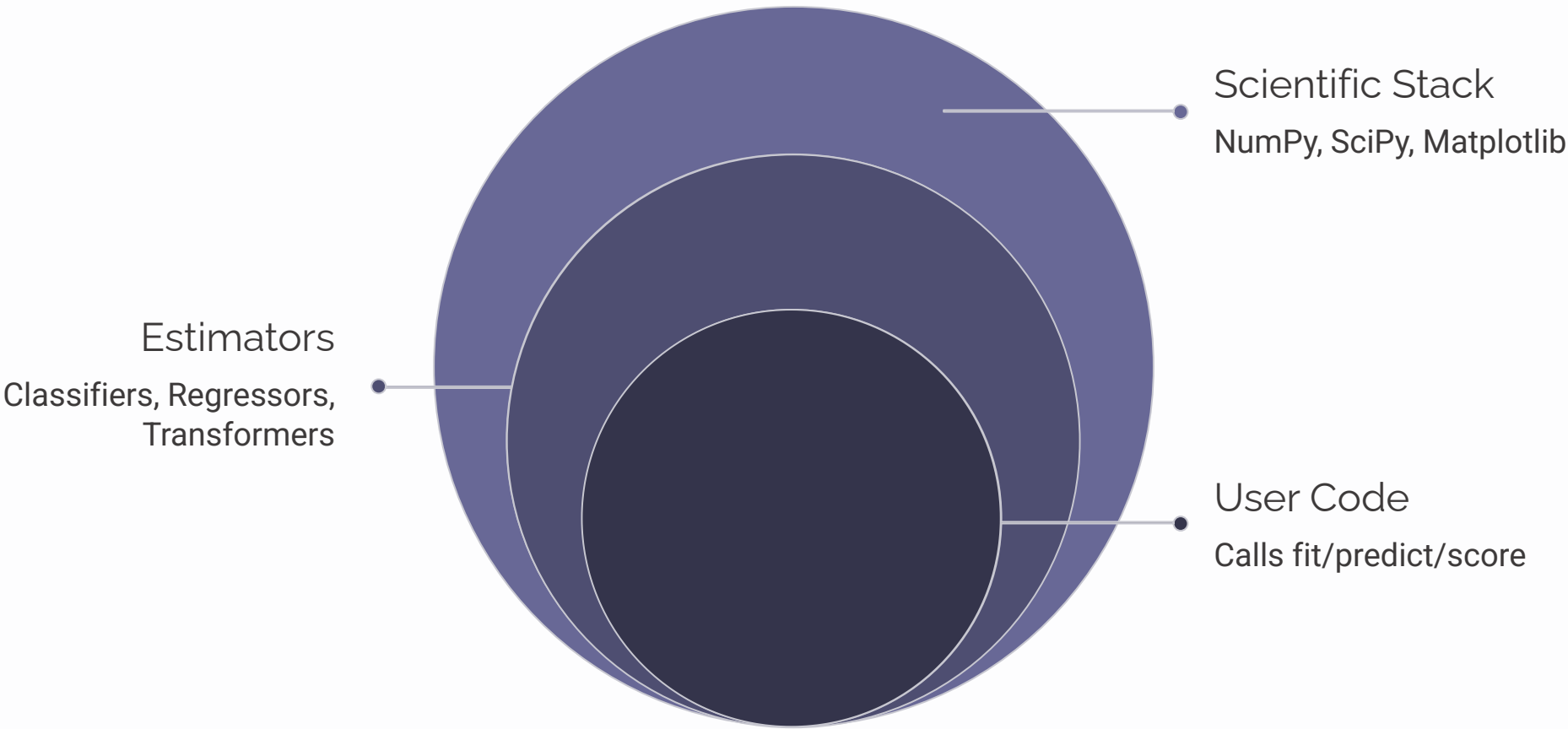
1. <code>fit()</code> Train the model on data	2. <code>predict()</code> Generate predictions
3. <code>score()</code> Evaluate performance	

Estimator API: All models are "Estimators" — they all behave identically regardless of the algorithm inside. Swap `RandomForest` for `SVM` by changing one line.

No GPU Required: Runs entirely on CPU — perfect for structured/tabular data. Deployable on any laptop or server without special hardware.

Key Modules — The Toolbox

<code>sklearn.preprocessing</code> Scale, encode, normalize data — <code>StandardScaler</code> , <code>OneHotEncoder</code>	<code>sklearn.model_selection</code> Train/test split, cross-validation, <code>GridSearchCV</code> hyperparameter tuning
<code>sklearn.linear_model</code> Linear Regression, Logistic Regression — interpretable baselines	<code>sklearn.ensemble</code> Random Forest, Gradient Boosting — high-accuracy production models
<code>sklearn.metrics</code> Accuracy, F1-score, confusion matrix, ROC-AUC	<code>sklearn.pipeline</code> Chain preprocessing + model into one reusable object



Scikit-learn in Python — Step-by-Step First Program

Problem Statement: Predict whether a student **passes or fails** based on study hours and attendance percentage. A classic binary classification problem — perfect for Logistic Regression.

1 Import Libraries

```
from sklearn.linear_model import LogisticRegression  
Also import train_test_split from sklearn.model_selection
```

2 Prepare Data

Features: `X = [[hours, attendance], ...]` | Labels: `y = [1=pass, 0=fail]`.
Ensure data is numerical — encode categories first.

3 Split Data

`train_test_split(X, y, test_size=0.2)` — 80% training, 20% testing. Random split prevents overfitting evaluation.

4 Create Model

`model = LogisticRegression()` — instantiate with default hyperparameters.
No configuration needed to start.

1 Train Model (The Learning Step)

`model.fit(X_train, y_train)` — model internally adjusts weights using gradient descent to minimise prediction error on training data.

2 Predict

`model.predict(X_test)` — applies learned weights to unseen test data. Returns array of class labels (0 or 1).

3 Evaluate

`model.score(X_test, y_test)` — returns accuracy as a percentage. Also use `classification_report()` for precision/recall/F1.

✅ **Key Insight:** These exact 7 steps work for ANY Scikit-learn algorithm. Only Step 4 changes — swap `LogisticRegression` for `RandomForestClassifier`, `SVM`, or any other estimator.

Scikit-learn Pipeline — The Professional Way to Code ML

The Problem Without Pipelines

Without pipelines, you must **manually apply preprocessing** to training data and then again — separately — to test data. This introduces a critical bug called **data leakage**: accidentally fitting the scaler on test data means your evaluation is unrealistically optimistic. Professionals never do manual preprocessing in production.

The Pipeline Solution

A Pipeline chains preprocessing and modelling into **ONE reusable object**. Call `fit()` once — it correctly fits the scaler on training data only and trains the model. Call `predict()` once — it automatically transforms test data through the scaler before passing to the model. Zero risk of leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

```
pipeline = Pipeline([
    ('scaler', StandardScaler()), # Step 1
    ('model', RandomForestClassifier()) # Step 2
])
```

```
pipeline.fit(X_train, y_train) # trains both
pipeline.predict(X_test)      # applies both
```

Pipeline Architecture Flow



Industrial Use: Pipelines are standard in production at Spotify (recommendation), Airbnb (pricing), and major banks (fraud detection).



PART 2 OF 4

TensorFlow

Google's Production Powerhouse — Deep Learning at Scale

From a 28×28 pixel image to a deployed REST API — TensorFlow covers the **entire production lifecycle**.

□ Dr. Anilkumar V. Brahmane | Computer Engineering Dept. | Sanjivani College of Engineering

TensorFlow Architecture — Understanding the Layers

Core Concept: What is a Tensor?

A **Tensor** is a multi-dimensional array — the generalisation of scalars, vectors, and matrices. A scalar is a 0D tensor. A vector is 1D. A matrix is 2D. An image batch is 4D (batch × height × width × channels). TensorFlow tensors behave like NumPy arrays but can run on GPUs/TPUs for massive speedups.

Computation Graph

TensorFlow models are represented as a **graph of mathematical operations**. Nodes = operations (add, matmul, relu). Edges = tensors flowing between them. In TF 2.x, **Eager Execution** runs operations immediately (like normal Python), making debugging intuitive. The XLA compiler then optimises the graph for hardware.

Ecosystem Tools

TensorBoard

Visualise training curves, model graphs, embeddings

TF Lite

Deploy on mobile and edge devices (Android, IoT)

TF Serving

Production REST/gRPC API server for models

TF.js

Run ML models directly in the browser

Architecture Layers (Top → Bottom)

1

2

3

4

1

● Hardware: CPU / GPU / TPU

NVIDIA CUDA GPUs for training. Google TPUs for massive scale. CPU for inference on servers.

2

● XLA Compiler

Optimises and fuses operations. Converts Python graph to machine-optimised code.

3

● TensorFlow Core

Low-level ops, custom training loops, gradient tape, automatic differentiation.

4

● Keras API

High-level model building: Sequential, Functional, Subclassing. Most users live here.

TensorFlow in Python — Building Your First Neural Network

Problem: Classify handwritten digits (0–9) using the **MNIST dataset** — 60,000 training images, 10,000 test images, each 28×28 pixels. The "Hello World" of deep learning. Target accuracy: ~98%.

01

Import TensorFlow

`import tensorflow as tf` — loads the complete TF ecosystem including Keras, NumPy utilities, and dataset loaders.

02

Load & Normalize Data

`mnist = tf.keras.datasets.mnist` then divide pixels by 255.0 — scales range from [0–255] to [0.0–1.0]. Neural networks train faster with normalised inputs.

03

Build Model (Sequential API)

`Flatten(28,28)` → converts 2D image to 784-element vector.
`Dense(128, activation='relu')` → hidden layer, learns patterns.
`Dense(10, activation='softmax')` → outputs 10 class probabilities.

04

Compile the Model

`optimizer='adam'` — adaptive learning rate optimiser.
`loss='sparse_categorical_crossentropy'` — standard for multi-class.
`metrics=['accuracy']` — track accuracy per epoch.

05

Train & Evaluate

`model.fit(x_train, y_train, epochs=5)` — 5 full passes through 60,000 images.
Then `model.evaluate(x_test, y_test)` — achieves **~98% accuracy** on 10,000 unseen images.

Neural Network Architecture: MNIST Classifier



Input Layer

784 nodes (28×28 pixels flattened). Each node holds one pixel value [0.0–1.0].



Hidden Layer

128 neurons, ReLU activation. Learns non-linear combinations of pixel patterns — edges, curves, loops.



Output Layer

10 neurons, Softmax activation. Each neuron represents one digit (0–9). Highest probability = predicted class.

60K

Training Images

~98%

Test Accuracy

5

Training Epochs



TensorFlow Keras — Three Ways to Build Models

1 Sequential API

Complexity: Beginner

Stack layers one after another in a linear chain. No branching, no multiple inputs/outputs. Perfect for learning and straightforward architectures.

```
model = tf.keras.Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

Use When: Building your first models, simple feedforward networks, quick prototypes.

2 Functional API

Complexity: Intermediate

Define inputs and outputs explicitly. Layers are called like Python functions. Supports branching, skip connections, multiple inputs/outputs — used in ResNet, Inception, U-Net.

```
inputs = tf.keras.Input(shape=(784,))
x = Dense(64, activation='relu')(inputs)
outputs = Dense(10, activation='softmax')(x)
model = tf.keras.Model(inputs, outputs)
```

Use When: Complex architectures with branching paths or multi-input/output models.

3 Model Subclassing

Complexity: Advanced

Write a custom Python class inheriting from `tf.keras.Model`. Define layers in `__init__` and the forward pass in `call()`. Maximum flexibility — any Python logic inside the model.

```
class MyModel(tf.keras.Model):
    def __init__(self): ...
    def call(self, x): ...
```

Use When: Research, non-standard architectures, custom training loops, publishing papers.

Rule of Thumb: Begin with Sequential API → graduate to Functional API when architecture requires branching → use Model Subclassing only for research-grade custom models. Most production models use the Functional API.

 **TensorBoard Tip:** Run `tensorboard --logdir logs/` to visualise training loss, accuracy curves, and the full model computation graph in your browser.

PART 3 OF 4

PyTorch

The Researcher's Favorite — Dynamic Graphs & Pythonic Design

Over **75% of AI research papers** use PyTorch. It powers GPT, LLaMA, BERT, and the models that define modern AI.

📄 Dr. Anilkumar V. Brahmane | Computer Engineering Dept. | Sanjivani College of Engineering



PyTorch Architecture — Dynamic Graphs & Autograd

Core Concept: Dynamic Computation Graph

PyTorch uses a "**Define-by-Run**" approach: the computation graph is built on-the-fly as your Python code executes. TensorFlow 1.x required building the entire graph before running (static graph). PyTorch's dynamic approach means you can use standard Python `if/else`, `for` loops, and recursion inside your model — the graph adapts each forward pass.

Why Dynamic Graphs Matter

Debugging is natural: use Python's standard `print()` and `pdb` debugger. Inspect any tensor mid-forward-pass. This makes PyTorch feel like writing regular Python — no special sessions, no graph compilation required. Researchers can try new model ideas in hours, not days.

Autograd Engine

PyTorch's **automatic differentiation engine** computes gradients (derivatives) for backpropagation automatically. Every tensor has a `requires_grad` flag. Set it to `True` — PyTorch tracks all operations. Call `loss.backward()` — gradients flow backwards through the computation graph to every parameter.

Architecture Modules

`torch.nn`

Neural network layers:
Linear, Conv2d, LSTM,
BatchNorm, Dropout — all
pre-built

`torch.optim`

Optimizers: SGD, Adam,
RMSprop, AdaGrad — plug-
and-play gradient descent
variants

`torch.autograd`

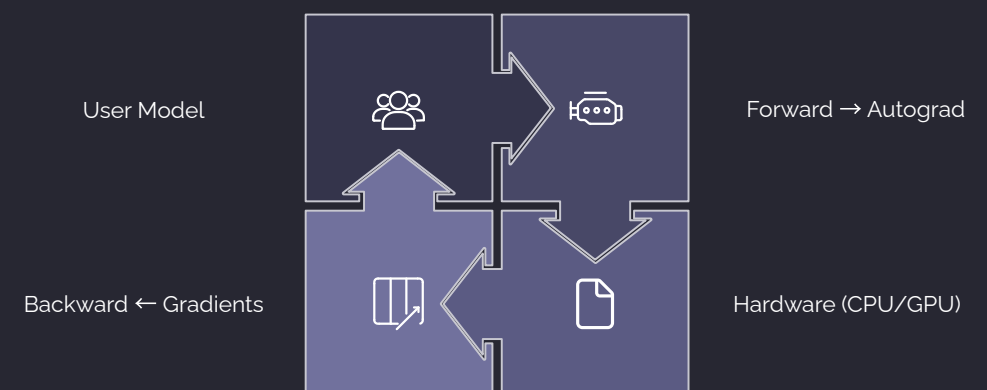
Automatic differentiation
engine — computes all
gradients via chain rule

`torch.utils.data`

DataLoader for batching,
shuffling, and parallel data
loading

`torchvision /` `torchaudio / torchtext`

Domain libraries — pre-
trained models, transforms,
dataset loaders



PyTorch in Python — Building a Neural Network from Scratch

Problem: Same MNIST digit classification as TensorFlow — but now we **write the training loop ourselves**. This exposes exactly what happens inside `model.fit()`, building deep intuition for backpropagation and gradient descent.

Model Definition & Setup

```
import torch
import torch.nn as nn
import torch.optim as optim

# Step 1: Define Model Class
class MNISTNet(nn.Module):
    def __init__(self):
        super().__init__()
        # Step 2: Define layers
        self.fc1 = nn.Linear(784, 128) # Input→Hidden
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10) # Hidden→Output

    def forward(self, x):
        x = self.relu(self.fc1(x)) # forward pass
        return self.fc2(x)

model = MNISTNet()

# Step 3: Loss Function
criterion = nn.CrossEntropyLoss()

# Step 4: Optimizer
optimizer = optim.Adam(
    model.parameters(), lr=0.001
)
```

The Training Loop — Step 5 (The Key Difference)

```
for epoch in range(5):
    for inputs, labels in train_loader:

        # Clear old gradients first
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)

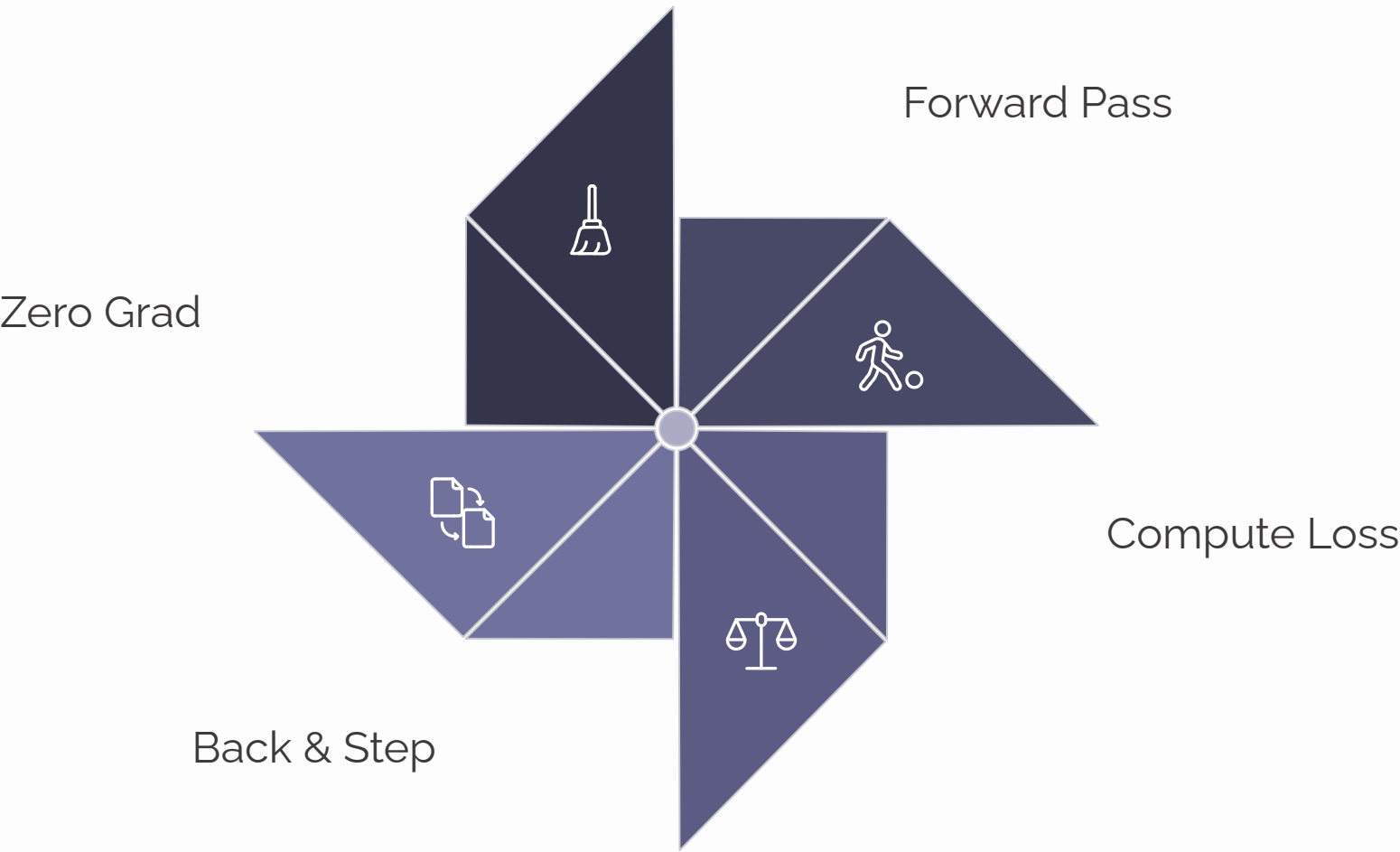
        # Compute loss (how wrong?)
        loss = criterion(outputs, labels)

        # Backprop: compute gradients
        loss.backward()

        # Update weights
        optimizer.step()

    print(f"Epoch {epoch}: Loss={loss:.4f}")
```

Why Write the Loop? Full control — print gradients mid-training, change learning rate per epoch, add custom regularisation, log any metric. TF's `model.fit()` hides all this.



PyTorch vs TensorFlow — Key Differences Explained Simply

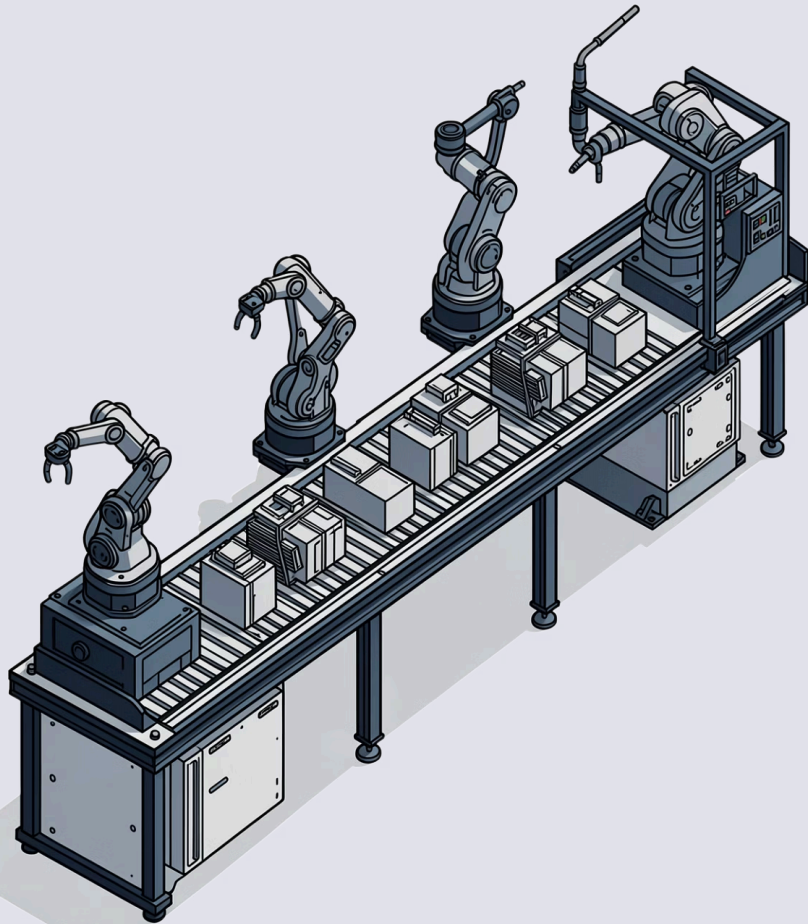
Feature	TensorFlow 2.x	PyTorch
Graph Type	Static + Eager Mode (TF2)	Dynamic — always (Define-by-Run)
Debugging	TensorBoard / tf.print()	Python print() / pdb — native
Training	model.fit() — automatic	Manual loop — full control
Deployment	TF Serving, TF Lite, TF.js — production-grade	TorchServe — growing ecosystem
Best For	Production, Mobile, Edge AI	Research, Experimentation
Popularity	Industry Leader (60%+ production systems)	Research Leader (75%+ papers)

When TensorFlow Wins

- Enterprise deployment with TF Serving REST APIs
- Mobile/edge deployment with TF Lite quantization
- Browser-based ML with TF.js
- Distributed training across multiple TPUs (Google Cloud)

When PyTorch Wins

- Implementing novel architectures from research papers
- Fine-tuning LLMs (GPT, LLaMA, BERT) — all PyTorch native
- Rapid experimentation requiring frequent architecture changes
- Academic research where paper reproducibility matters



PART 4 OF 4

Industrial Applications & Framework Selection — Real- World Deployment

How leading companies across industries map problems to the **right framework** at the right scale.

□ Dr. Anilkumar V. Brahmane | Computer Engineering Dept. | Sanjivani College of Engineering

Industrial Applications — Which Framework for Which Problem?

Industry	Problem	Best Framework	Algorithm / Model
 Healthcare	Disease Risk Prediction	Scikit-learn	Random Forest
 Finance	Fraud Detection	Scikit-learn	Gradient Boosting
 Manufacturing	Defect Detection (Edge)	TensorFlow Lite	CNN
 E-Commerce	Recommendation Systems	TensorFlow	Deep Neural Net
 NLP / AI	Chatbots / LLMs	PyTorch	Transformers
 Autonomous Cars	Object Detection	PyTorch → TF	YOLO / ResNet

Healthcare Deep Dive

Scikit-learn handles patient risk scoring on tabular EHR data (age, BP, glucose, BMI). Interpretable models (Logistic Regression, Decision Trees) satisfy clinical audit requirements. TensorFlow/PyTorch handle medical image analysis — chest X-ray pneumonia detection, MRI tumour segmentation — achieving radiologist-level accuracy.

Manufacturing Deep Dive

PyTorch is used in the lab to train CNN models on labelled defect images. The trained model is then converted to **TensorFlow Lite** format and deployed on edge cameras mounted on assembly lines. Real-time inference at the factory edge — no cloud latency. Companies report 40–60% reduction in defect escape rates.

NLP / LLM Deep Dive

Modern Large Language Models (GPT-4, LLaMA-3, Mistral) are all trained and fine-tuned in **PyTorch**. Google's BERT powering Search was originally TensorFlow; Hugging Face Transformers supports both. The industry pattern: research in PyTorch → convert to TF/ONNX → deploy in production.

How to Choose the Right Framework — Decision Guide

✓ Use Scikit-learn When...

- Data is tabular — rows and columns like Excel/CSV
- You need quick results and model interpretability
- Problem is classification, regression, or clustering
- No GPU available — any laptop works
- Need explainability for stakeholders (feature importance)

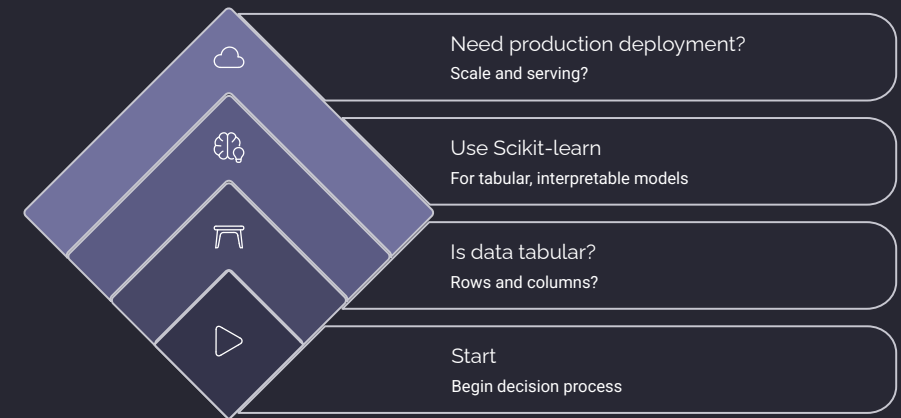
✓ Use TensorFlow When...

- Deploying to mobile (TF Lite), browser (TF.js), or production server (TF Serving)
- Working in a company or enterprise environment
- Need distributed training across multiple GPUs/TPUs
- Integrating with Google Cloud AI Platform

✓ Use PyTorch When...

- Doing research or building custom/novel architectures
- Working with images, text, or audio (CV, NLP, ASR)
- Need maximum flexibility and Python-native debugging
- Implementing the latest AI research papers (75%+ use PyTorch)

Framework Decision Tree



Pro Tip from the Field: In real industry projects, you often use **ALL THREE** together: Scikit-learn for feature engineering and preprocessing, PyTorch for training the deep model, TensorFlow for production deployment. There is no "best" framework — only "best for your situation."

Putting It All Together — A Complete ML Project Workflow

Golden Rule: Always start with the simplest solution. Add complexity only when it demonstrably improves results. A well-tuned Random Forest often beats a poorly-tuned neural network.

1

Phase 1: Data Prep (*Scikit-learn*)

Load CSV with Pandas. Handle missing values: `SimpleImputer`. Scale features: `StandardScaler`. Encode categories: `OneHotEncoder`. Build a Pipeline — no data leakage.

2

Phase 2: Baseline Model (*Scikit-learn*)

Try `RandomForestClassifier` first — typically achieves 80–90% accuracy on tabular data. Evaluate with cross-validation. **Decision:** If accuracy satisfies requirements → deploy now. If not → proceed to deep learning.

3

Phase 3: Deep Learning (*PyTorch*)

Build custom `nn.Module`. Experiment with architecture: layers, dropout, batch normalisation. Use `DataLoader` for batching. Track experiments. Iterate rapidly using Pythonic debugging.

4

Phase 4: Deploy (*TensorFlow*)

Convert model to TensorFlow SavedModel format. Deploy via **TF Serving** as REST API. Optimise with TF Lite for edge. Monitor live metrics with TensorBoard.

The Decision Point Between Phase 2 and Phase 3

After building the Scikit-learn baseline, ask: "**Does accuracy meet the business requirement?**" If a Random Forest at 87% accuracy is sufficient for a fraud detection use case, stop there — it is faster, cheaper, more interpretable, and easier to maintain than a deep learning model. Only escalate to PyTorch/TensorFlow when the problem genuinely requires the additional complexity: unstructured data (images, text, audio), very large datasets, or accuracy requirements above 95%+.

80%

Problems Solved by Scikit-learn Alone

3

Frameworks in a Typical Production Pipeline

Key Takeaways & Next Steps for B.Tech Students

The Three-Framework Mastery Path



Master Scikit-learn First

Learn `fit()`, `predict()`, `Pipeline` cold. This single framework solves **80% of real ML problems** in industry. Random Forest and Gradient Boosting are workhorses at every major tech company.



Learn PyTorch Second

Understand tensors, autograd, and writing your own training loop. This builds **deep intuition** for how neural networks actually work — gradient descent, backpropagation, weight updates. Essential for AI research roles.



Add TensorFlow for Deployment

Learn Keras Sequential API and TF Serving. **Essential for industry jobs** — 60%+ of production ML systems are TF-based. TF Lite for mobile, TF Serving for APIs.



Industry Demand 2026: ML Engineer roles require proficiency in at least 2 of these 3 frameworks. All three is a strong differentiator in campus placements and industry hiring.

12-Week Learning Roadmap

Weeks 1–2:
Foundation

1

Python + NumPy +
Pandas basics.
Master array
operations,
DataFrames, data
visualisation with
Matplotlib.

2

Weeks 3–4:
Scikit-learn

Classification,
regression,
clustering. Pipelines,
cross-validation,
GridSearchCV.

**Milestone: First ML
Model**

Weeks 5–8:
PyTorch

3

Tensors, nn.Module,
training loops, CNNs.
Train MNIST, then
CIFAR-10. **Milestone:
First Neural Network**

4

Weeks 9–12:
TensorFlow +
Deploy

Keras APIs, transfer
learning, TF Serving
REST API.

**Milestone: First
Deployed App**



References & Sources

2025–2026 Curated Reading List

Python ML Frameworks Compared (2025)

Scikit-learn, TensorFlow, and PyTorch – a practical comparison covering use cases, APIs, and selection criteria for production systems.

[dataa.dev – C4: Container, Code, Cloud & Context](#)

TensorFlow vs. PyTorch vs. Scikit-Learn (2025)

QualityPoint Technologies' detailed breakdown of architectural differences, learning curves, and enterprise deployment considerations.

[blog.qualitypointtech.com](#)

Best Machine Learning Frameworks in 2024: A Deep Dive

Comprehensive analysis of framework performance benchmarks, community size, and industry adoption rates across sectors.

[DeepUseCase.com](#)

TensorFlow vs PyTorch vs Scikit-learn: Best ML Framework 2026

Forward-looking analysis covering emerging trends, edge AI deployment, and framework roadmaps for 2025–2026.

[AISTechnoLabs.com](#)

TensorFlow vs PyTorch for Manufacturing

Industry-specific comparison for manufacturing use cases including quality control, predictive maintenance, and edge deployment on factory floors.

[Qualitech.ai](#)

